

PES-VCS Lab Report

Student Details

- **Name:** Pallavi B
 - **SRN:** PES1UG24AM906
 - **Assignment:** Building PES-VCS (Version Control System)
-

Objective

To design and implement a simplified version control system similar to Git, supporting object storage, staging, commits, and history tracking using low-level filesystem concepts.

Environment Setup

- OS: Ubuntu 22.04 (WSL)
- Compiler: GCC
- Libraries Used: OpenSSL (for SHA-256)

Installation Commands:

```
sudo apt update
```

```
sudo apt install gcc build-essential libssl-dev
```

Phase 1: Object Storage

Objective

Implement content-addressable storage using SHA-256 hashing.

✂ Implementation

- Implemented `object_write()`:
 - Created header (blob <size>\0)
 - Computed SHA-256 hash
 - Stored objects in `.pes/objects/XX/YYY...`
- Implemented `object_read()`:
 - Parsed stored object
 - Verified hash integrity

Output – 1A & 1B

- Successfully stored and retrieved objects

- Deduplication verified

```

root@pallavib: ~/PES1UG24AI
(Reading database ... 49013 files and directories currently installed.)
Preparing to unpack .../openssl_3.0.13-0ubuntu3.9_amd64.deb ...
Unpacking openssl (3.0.13-0ubuntu3.9) over (3.0.13-0ubuntu3.7) ...
Setting up libssl-dev:amd64 (3.0.13-0ubuntu3.9) ...
Setting up openssl (3.0.13-0ubuntu3.9) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
root@pallavib:~# git clone https://github.com/pallaviB463/PES1UG24AM906-pes-vcs.git
Cloning into 'PES1UG24AM906-pes-vcs'...
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 17 (delta 0), reused 11 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), 24.79 KiB | 317.00 KiB/s, done.
root@pallavib:~# cd PES1UG24AM906-pes-vcs
root@pallavib:~/PES1UG24AM906-pes-vcs# export PES_AUTHOR="PES1UG24AM906_PallaviB"
root@pallavib:~/PES1UG24AM906-pes-vcs# make
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
root@pallavib:~/PES1UG24AM906-pes-vcs# nano object.c
root@pallavib:~/PES1UG24AM906-pes-vcs# make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
root@pallavib:~/PES1UG24AM906-pes-vcs# ./test_objects
Stored blob with hash: d58213f5db0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5db0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
root@pallavib:~/PES1UG24AM906-pes-vcs# find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f880dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5db0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
root@pallavib:~/PES1UG24AM906-pes-vcs#

```

Phase 2: Tree Objects

Objective

Represent directory structures using tree objects.

🔧 Implementation

- Implemented `tree_from_index()`:
 - Parsed file paths
 - Built hierarchical tree structure
 - Stored tree objects using object storage

Output - 2A & 2B

- Tree objects created correctly
- Serialization and parsing verified

```

root@pallavib:~/PES1UG24AM906-pes-vcs# ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
root@pallavib:~/PES1UG24AM906-pes-vcs# find .pes/objects -type f
root@pallavib:~/PES1UG24AM906-pes-vcs#

```

```

root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes init
Initialized empty PES repository in .pes/
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "hello" > a.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "world" > b.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes add a.txt b.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes commit -m "test commit"
Committed as cc71e3e89f986482282f5e8b3c09354049de0392ea4589910fbd124df3e4d00a
Committed: cc71e3e89f98... test commit
root@pallavib:~/PES1UG24AM906-pes-vcs# find .pes/objects -type f
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/cc/71e3e89f986482282f5e8b3c09354049de0392ea4589910fbd124df3e4d00a
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/5b/0410c33d2229bbe377a506f669680429c80cb9fa70859a576f8463633e27cf
root@pallavib:~/PES1UG24AM906-pes-vcs# xxd .pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f | head -20
00000000: 626c 6f62 2036 0077 6f72 6c64 0a          blob 6.world.
root@pallavib:~/PES1UG24AM906-pes-vcs#

```

Phase 3: Index (Staging Area)

Objective

Track staged files before committing.

❌ Implementation

- `index_load()`:
 - Reads `.pes/index`
 - Initializes empty index if file missing
- `index_add()`:
 - Computes file hash
 - Updates index entries
- `index_save()`:
 - Writes sorted entries atomically

Output-3A & 3B

`./pes status`

- Shows staged, unstaged, and untracked files correctly

```

root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes init
Initialized empty PES repository in .pes/
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "hello" > file1.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "world" > file2.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes add file1.txt file2.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes status
Staged changes:
  staged:   file1.txt
  staged:   file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: pes.c
  untracked: test_tree.c
  untracked: Makefile
  untracked: test_sequence.sh
  untracked: commit.c
  untracked: commit.h
  untracked: tree.c
  untracked: pes.h
  untracked: tree.h
  untracked: README.md
  untracked: index.c
  untracked: object.c
  untracked: .gitignore
  untracked: index.h
  untracked: test_objects.c

root@pallavib:~/PES1UG24AM906-pes-vcs# cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776774806 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776774819 6 file2.txt
root@pallavib:~/PES1UG24AM906-pes-vcs#

```

Phase 4: Commit System

Objective

Create snapshots and maintain commit history.

✂ Implementation

- Implemented `commit_create()`:
 - Generated tree from index
 - Linked parent commit
 - Stored commit object
 - Updated HEAD

Output- 4A,4B & 4C

`./pes log`

- Displays commit history correctly

```
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes init
Initialized empty PES repository in .pes/
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "Hello" > hello.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes add hello.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes commit -m "Initial commit"
Committed as 1ccf36243a0db4286227e8b68969eccadc26067360ffeb88bec5565d18050ca7
Committed: 1ccf36243a0d... Initial commit
root@pallavib:~/PES1UG24AM906-pes-vcs# echo "World" >> hello.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes add hello.txt
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes commit -m "Second commit"
Committed as e55c600f236e76d7304453c97aff007f5274d7155644ebcbab509307e63cf6cb
Committed: e55c600f236e... Second commit
root@pallavib:~/PES1UG24AM906-pes-vcs# ./pes log
commit e55c600f236e76d7304453c97aff007f5274d7155644ebcbab509307e63cf6cb
Author: PES1UG24AM906_PallaviB
Date: 1776776025

    Second commit

commit 1ccf36243a0db4286227e8b68969eccadc26067360ffeb88bec5565d18050ca7
Author: PES1UG24AM906_PallaviB
Date: 1776775931

    Initial commit

root@pallavib:~/PES1UG24AM906-pes-vcs# find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/1c/cf36243a0db4286227e8b68969eccadc26067360ffeb88bec5565d18050ca7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e5/5c600f236e76d7304453c97aff007f5274d7155644ebcbab509307e63cf6cb
.pes/refs/heads/main
root@pallavib:~/PES1UG24AM906-pes-vcs# cat .pes/Head
cat: .pes/Head: No such file or directory
root@pallavib:~/PES1UG24AM906-pes-vcs# cat .pes/HEAD
ref: refs/heads/main
root@pallavib:~/PES1UG24AM906-pes-vcs# cat .pes/refs/heads/main
e55c600f236e76d7304453c97aff007f5274d7155644ebcbab509307e63cf6cb
root@pallavib:~/PES1UG24AM906-pes-vcs# |
```

Integration Test

make test-integration

- All components worked together successfully

```

root@pallavib:~/PES1UG24AM906-pes-vcs# make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed as 37451284b9d543ec4dde99d0aaa6f217b7c5858ddc4cd2bae0215b6777808180
Committed: 37451284b9d5... Initial commit

Log after first commit:
commit 37451284b9d543ec4dde99d0aaa6f217b7c5858ddc4cd2bae0215b6777808180
Author: PES1UG24AM906_PallaviB
Date: 1776776259

    Initial commit

--- Second Commit ---
Committed as 3f9309e536e1f11265b1a113ae6fcae3ef2ec49d13cf94eae9804a3f8991c33
Committed: 3f9309e536e1... Update file.txt

--- Third Commit ---
Committed as a7aa39cc0bdc5239d655c7b8a0ae3ea6e269cbb07d967320b1d1f14d79ce31a9
Committed: a7aa39cc0bdc... Add farewell

--- Full History ---

```

```

Committed as a7aa39cc0bdc5239d655c7b8a0ae3ea6e269cbb07d967320b1d1f14d79ce31a9
Committed: a7aa39cc0bdc... Add farewell

--- Full History ---
commit a7aa39cc0bdc5239d655c7b8a0ae3ea6e269cbb07d967320b1d1f14d79ce31a9
Author: PES1UG24AM906_PallaviB
Date: 1776776259

    Add farewell

commit 3f9309e536e1f11265b1a113ae6fcae3ef2ec49d13cf94eae9804a3f8991c33
Author: PES1UG24AM906_PallaviB
Date: 1776776259

    Update file.txt

commit 37451284b9d543ec4dde99d0aaa6f217b7c5858ddc4cd2bae0215b6777808180
Author: PES1UG24AM906_PallaviB
Date: 1776776259

    Initial commit

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
a7aa39cc0bdc5239d655c7b8a0ae3ea6e269cbb07d967320b1d1f14d79ce31a9

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ae3c4c7cbfa
.pes/objects/37/451284b9d543ec4dde99d0aaa6f217b7c5858ddc4cd2bae0215b6777808180
.pes/objects/3f/9309e536e1f11265b1a113ae6fcae3ef2ec49d13cf94eae9804a3f8991c33
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/a7/aa39cc0bdc5239d655c7b8a0ae3ea6e269cbb07d967320b1d1f14d79ce31a9
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
root@pallavib:~/PES1UG24AM906-pes-vcs# |

```

Analysis Questions

Q5.1: How would you implement pes checkout <branch>?

A branch in Git is stored as a file inside `.pes/refs/heads/` containing a commit hash. To implement pes checkout <branch>, the following steps are required:

1. Update .pes/HEAD

- Modify the `.pes/HEAD` file to point to the new branch:
- `ref: refs/heads/<branch>`

2. Read Target Commit

- Open `.pes/refs/heads/<branch>` and read the commit hash.
- Load the commit object using `object_read()`.

3. Load Tree Structure

- Extract the tree hash from the commit.
- Recursively read tree objects to reconstruct directory structure.

4. Update Working Directory

- Replace current files with files from the target tree:
 - Create directories as needed
 - Write blob contents into files
 - Remove files that do not exist in the target tree

5. Update Index

- Clear current index
- Populate index with files from the new tree

Why is checkout complex?

- Must safely overwrite working directory files
 - Must detect uncommitted changes and avoid data loss
 - Requires recursive traversal of tree objects
 - Needs to handle file deletions and directory changes
 - Must maintain consistency between index and working directory
-

Q5.2: Detecting a "dirty working directory"

To detect uncommitted changes:

Step 1: Compare Index with Working Directory

For each entry in the index:

- Check file metadata:
 - Modification time (mtime)
 - File size
- If mismatch → file is modified

Step 2: Compare with Object Store

- Retrieve stored blob using hash
- Compute hash of current file
- If hashes differ → file content changed

Step 3: Detect Conflict

- If a tracked file is modified AND
- That file differs in target branch
→ checkout must be refused

Q5.3: Detached HEAD

Detached HEAD occurs when:

- .pes/HEAD contains a commit hash directly instead of a branch reference

What happens when committing?

- New commits are created normally
- But they are **not referenced by any branch**
- These commits can become unreachable

How to recover commits?

- Create a new branch pointing to that commit:
- `pes branch new_branch <commit_hash>`
- Or manually update a ref file

Garbage Collection and Space Reclamation

Q6.1: Finding and deleting unreachable objects

Algorithm:

1. **Start from all branch heads**
 - Read all files in .pes/refs/heads/
2. **Traverse commit graph**
 - For each commit:
 - Mark commit as reachable
 - Visit parent commits
 - Visit associated tree
3. **Traverse tree objects**
 - For each tree:
 - Mark as reachable
 - Visit child blobs and subtrees
4. **Mark reachable objects**
 - Store all visited hashes
5. **Delete unreachable objects**
 - Iterate through .pes/objects/
 - Delete objects not in reachable set

Data Structure:

- Use a **Hash Set** (or hash table)
 - Fast lookup: $O(1)$
 - Efficient for large number of objects

Estimated objects visited:

For:

- 100,000 commits
- 50 branches

You may visit:

- All commits ($\approx 100,000$)
- Trees per commit (could be multiple)
- Blobs referenced by trees

Total objects visited: **hundreds of thousands to millions**

Q6.2: Why GC is dangerous during commit

Race condition:

1. Commit process:
 - Writes blob → tree → commit object
 - Has not yet updated HEAD
 2. GC runs at same time:
 - Scans reachable objects
 - New objects are not yet referenced
 - GC deletes them
 3. Commit finishes:
 - HEAD now points to missing objects → corruption
-

How Git avoids this:

- Uses **temporary references**
 - Avoids deleting recent objects
 - Uses **locks** during critical operations
 - Runs GC only when repository is stable
-

Conclusion

These concepts highlight the complexity of version control systems, especially:

- Safe file updates
 - Consistency guarantees
 - Efficient storage management
-